第9章作业　9.3　9.31　9.33

第9章　查找

9.1 基本概念

9.2 静态查找表

1、顺序查找（线性查找）
2、折半查找（二分或对分查找）
3、分块查找（索引顺序查找）
4、静态树表的查找（次优树）

9.3 动态查找表

二叉排序树

9.4 哈希查找表

教材第8、11和12章省略，因《操作系统》课程会涉及。

怎么样能最高效的查找？

- 我们已经学了，如果关键字是有序的，那么可以在log(n)时间内找到
- 下一章将学到，如果要进行排序，我们最优可以做到nlog(n)，因此对于先排序后查找来讲，最优的复杂度是nlog(n)，
- 而且数学上已经证明，任何基于比较的排序算法，复杂度不可能低于nlog(n)
- 这么说无论怎么样改进，查找的最低复杂度为nlog(n)，这个结论正确吗？

不进行比较是不是能更快？

- 既然基于比较的算法复杂度已经达到下限，那么要进一步改进，只能是放弃“比较”这个操作
- 不进行比较怎么能找到数据呢？
- 换一个思路，我们不通过比较找到需要的数据，而是直接根据关键字计算数据所在的地址，那么同样能找到数据
- 这样的方法就是哈希！

9.4 哈希查找表

是一种查找效率高达
O(1)的算法!

一、哈希表的概念

二、哈希函数的构造方法

三、冲突处理方法

四、哈希表的查找及分析

一、哈希表的概念

哈希表：**即散列存储结构。**

散列法存储的基本思想：**建立关键码字与其存储位置的**对应关系，**或者说，由关键码的值决定数据的存储地址。**

优点：**查找速度极快（O(1)）,查找效率与元素个数n无关！**

例1：若将学生信息按如下方式存入计算机，如：
将200101181020 1的所有信息存入V[01]单元；
将200101181020 2的所有信息存入V[02]单元；
……
将200101181023 1的所有信息存入V[31]单元。

欲查找学号为2001011810216的信息，便可直接访问V[16]！

例2：有数据元素序列(14，23，39，9，25，11)，若规定每个元素k的存储地址H（k）＝k，请画出存储结构图。

H(k)称为散列函数

解：根据散列函数H（k）＝k，可知元素14应当存入地址为14的单元，元素23应当存入地址为23的单元，……，

对应散列存储表（哈希表）如下：

地址	…	9	…	11	…	14	…	23	24	25	…	39	…
内容		9		11		14		23		25		39	

讨论：如何进行散列查找？

根据存储时用到的散列函数H(k)表达式，迅即可查到结果！

例如，查找key=9,则访问H(9)=9号地址，若内容为9则成功；若查不到，应当设法返回一个特殊值，例如空指针或空记录。

明显缺点：空间效率低

如何解决?

Hash查找法

用尽量少的冗余空间实现O(1)的查找效率

哈希方法(杂凑法)

选取某个函数，依该函数按关键字计算元素的存储位置并按此存放；查找时也由同一个函数对给定值k计算地址，将k与地址中内容进行比较，确定查找是否成功。

哈希函数(杂凑函数)

哈希方法中使用的转换函数称为哈希函数(杂凑函数)

哈希表(杂凑表)

按上述思想构造的表称为哈希表(杂凑表)

冲 突

通常关键码的集合比哈希地址集合大得多，因而经过哈希函数变换后，可能将不同的关键码映射到同一个哈希地址上，这种现象称为冲突。

冲突现象举例：

有6个元素的关键码分别为：（14，23，39，9，25，11）。
选取关键码与元素位置间的函数为H(k)=k mod 7

6个元素用7个地址应该足够!

通过哈希函数对6个元素建立哈希表：

0	1	2	3	4	5	6
14		23		39		
		9		25		
				11		

H(14)=14%7=0

H(25)=25%7=4
H(11)=11%7=4

有冲突!

在哈希查找方法中，冲突是不可能避免的，只能尽可能减少。

所以，哈希方法必须解决以下两个问题：

1）构造好的哈希函数

（a）所选函数尽可能简单，以便提高转换速度；

（b）所选函数对关键码计算出的地址，应在哈希地址内集中并大致均匀分布，以减少空间浪费。

2）制定一个好的解决冲突的方案

查找时，如果从哈希函数计算出的地址中查不到关键码，则应当依据解决冲突的规则，有规律地查询其它相关单元。

二、哈希函数的构造方法

要求一：n个数据原仅占用n个地址，虽然散列查找是以空间换时间，但仍希望散列的地址空间尽量小。

要求二：无论用什么方法存储，目的都是尽量均匀地存放元素，以避免冲突。

常用的哈希函数构造方法有：

1. 直接定址法
2. 除留余数法
3. 乘余取整法
4. 数字分析法
5. 平方取中法
6. 折叠法
7. 随机数法

1、直接定址法

Hash(key) = a·key + b　(a、b为常数)

优点：以关键码key的某个线性函数值为哈希地址，不会产生冲突.

缺点：要占用连续地址空间，空间效率低。

例：关键码集合为{100，300，500，700，800，900}，选取哈希函数为Hash(key)=key/100，则存储结构（哈希表）如下：

0	1	2	3	4	5	6	7	8	9
	100		300		500		700	800	900

2、除留余数法

Hash(key)=key mod p　(p是一个整数)

特点：以关键码除以p的余数作为哈希地址。

关键：如何选取合适的p?

技巧：若设计的哈希表长为m，则一般取p≤m且为质数（也可以是合数，但不能包含小于20的质因子）。

3、乘余取整法

Hash(key)=⌊B*(A*key mod 1)⌋

(A*key mod 1)就是取A*key 的小数部分

(A、B均为常数，且0<A<1，B为整数)

特点：以关键码key乘以A，取其小数部分，然后再放大B倍并取整，作为哈希地址。

例：欲以学号最后两位作为地址，则哈希函数应为：

H(k)=100*(0.01*k % 1)

其实也可以用法2实现：　H(k)=k % 100

4、数字分析法

特点：选用关键字的某几位组合成哈希地址。选用原则应当是：各种符号在该位上出现的频率大致相同。

例：有一组（例如80个）关键码，其样式如下：

3	4	7	0	5	2	4
3	4	9	1	4	8	7
3	4	8	2	6	9	6
3	4	8	5	2	7	0
3	4	8	6	3	0	5
3	4	9	8	0	5	8
3	4	7	9	6	7	1
3	4	7	3	9	1	9
位号：	①	②	③	④	⑤	⑥ ⑦

讨论：

① 第1、2位均是“3和4”，第3位也只有“7、8、9”，因此，这几位不能用，余下四位分布较均匀，可作为哈希地址选用。

② 若哈希地址取两位（因元素仅80个），则可取这四位中的任意两位组合成哈希地址，也可以取其中两位与其它两位叠加求和后，取低两位作哈希地址。

5、平方取中法

特点：对关键码平方后，按哈希表大小，取中间的若干位作为哈希地址。

理由：因为中间几位与数据的每一位都相关。

例：2589的平方值为6702921，可以取中间的029为地址。

6、折叠法

特点：将关键码自左到右分成位数相等的几部分（最后一部分位数可以短些），然后将这几部分叠加求和，并按哈希表表长，取后几位作为哈希地址。

适用于：每一位上各符号出现概率大致相同的情况。

法1：移位法 —— 将各部分的最后一位对齐相加。

法2：间界叠加法——从一端向另一端沿分割界来回折叠后，最后一位对齐相加。

例：元素42751896, 用法1： 427 + 518 + 96 = 1041

用法2： 427 518 96 —> 724 + 518 + 69 = 1311

7、随机数法

Hash(key) = random (key) (random为伪随机函数)

适用于：关键字长度不等的情况。造表和查找都很方便。

小结：构造哈希函数的原则：

① 执行速度（即计算哈希函数所需时间）；

② 关键字的长度；

③ 哈希表的大小；

④ 关键字的分布情况；

⑤ 查找频率。

三、冲突处理方法

常见的冲突处理方法有：

1. **开放定址法（开地址法）**
2. **链地址法（拉链法）**
3. **再哈希法（双哈希函数法）**
4. **建立一个公共溢出区**

1、开放定址法（开地址法）

设计思路： 有冲突时就去寻找下一个空的哈希地址，只要哈希表足够大，空的哈希地址总能找到，并将数据元素存入。

具体实现：

1）线性探测法

$$H_i=(Hash(key)+d_i) \bmod m \qquad (1 \le i < m)$$

其中：

Hash(key)为哈希函数

m为哈希表长度

d_i 为增量序列 1，2，…m-1，且$d_i=i$

含义：一旦冲突，就找附近（下一个）空地址存入。

例： 关键码集为 {47，7，29，11，16，92，22，8，3}，

设：哈希表表长为m=11；

哈希函数为Hash(key)=key mod 11；

拟用线性探测法处理冲突。建哈希表如下：

0	1	2	3	4	5	6	7	8	9	10
11	22		47	92	16	3	7	29	8	
	△					▲		△	△	

解释：

① 47、7是由哈希函数得到的没有冲突的哈希地址；

② Hash(29)=7，哈希地址有冲突，需寻找下一个空的哈希地址：由H_1=(Hash(29)+1) mod 11=8，哈希地址8为空，因此将29存入。

③ 另外，22、8、3同样在哈希地址上有冲突，也是由H_1找到空的哈希地址的。

其中3 还连续移动了两次（二次聚集）

讨论：

线性探测法的优点：只要哈希表未被填满，保证能找到一个空地址单元存放有冲突的元素；

线性探测法的缺点：可能使第i个哈希地址的同义词存入第i+1个哈希地址，这样本应存入第i+1个哈希地址的元素变成了第i+2个哈希地址的同义词，……，

因此，可能出现很多元素在相邻的哈希地址上“堆积”起来，大大降低了查找效率。

解决方案：可采用二次探测法或伪随机探测法，以改善“堆积”问题。

2） 二次探测法

仍举上例，改用二次探测法处理冲突，建表如下：

$H_i=(Hash(key)\pm d_i) \bmod m$

其中：Hash(key)为哈希函数

m为哈希表长度，m要求是某个4k+3的质数；

d_i为增量序列 1^2，-1^2，2^2，-2^2，…，q^2

0	1	2	3	4	5	6	7	8	9	10
11	22	3	47	92	16		7	29	8	
	△	▲						△	△	

注：只有3这个关键码的冲突处理与上例不同，

Hash(3)=3，哈希地址上冲突，由

$H_1=(Hash(3)+1^2) \bmod 11=4$，仍然冲突；

$H_2=(Hash(3)-1^2) \bmod 11=2$，找到空的哈希地址，存入。

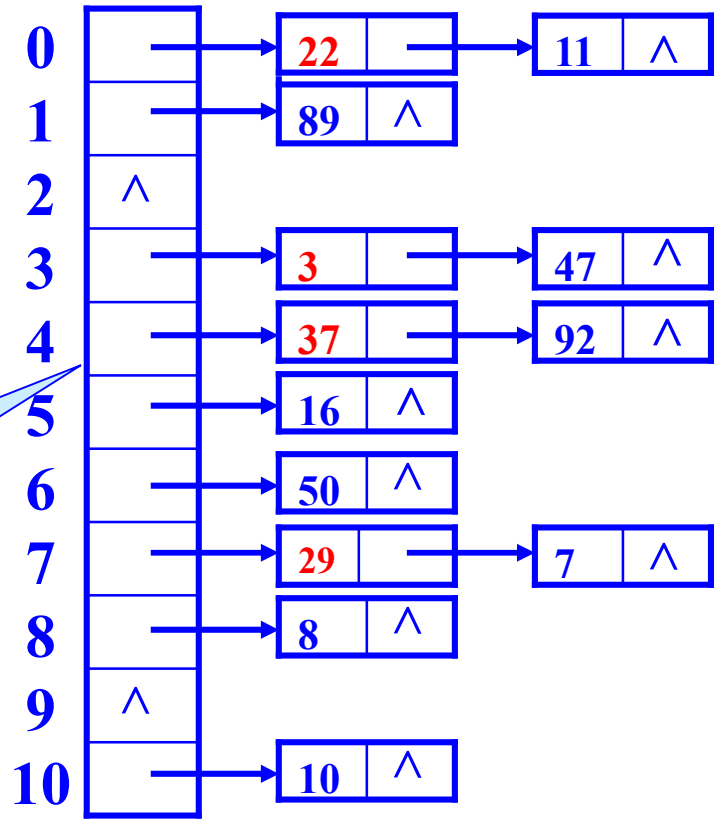
3） 若di = 伪随机序列，就称为伪随机探测法

2、链地址法(拉链法)

基本思想：将具有相同哈希地址的记录链成一个单链表，m个哈希地址就设m个单链表，然后用一个数组将m个单链表的表头指针存储起来，形成一个动态的结构。

例：设{ 47, 7, 29, 11, 16, 92, 22, 8, 3, 50, 37, 89 }的哈希函数为：Hash(key)=key mod 11，用拉链法处理冲突，则建表如右图所示。

有冲突的元素可以插在表尾,也可以插在表头。



严蔚敏习题集9.45③　假设哈希表长为*m*，哈希函数为H(*x*)，用链地址法处理冲突。试编写输入一组关键字并建造哈希表的算法。

```
Typedef *LNode[MAXSIZE] CHashTable; //链地址Hash表类型
  Status Build_Hash (CHashTable &T, int m)
//输入一组关键字,建立Hash表，表长为m，用链地址法处理冲突
  {
   if(m<1) return ERROR;
   T=malloc(m*sizeof(WORD));   //开表空间，建立表头指针向量
   for(i=0; i<m; i++) T[i]=NULL;  //哈希表初始化
```

```
    while( (key=Inputkey())!=NULL )
//假定Inputkey函数用于从键盘输入关键字
 {
      q=(LNode*)malloc(sizeof(LNode));
  q->data=key; q->next=NULL;
      n=H(key);
      if(!T[n]) T[n]=q; //若无冲突，则作为链表的第一个结点
      else
   {
    for( p=T[n]; p->next; p=p->next);
    p->next=q;  //有冲突则插入链表尾部
   }
     }//while
    return OK;
}//Build_Hash
```

若当前结点不是末结点则继续“摸瓜”

通信0703班霍崇同学的头插入算法：

```
while( (key=Inputkey())!=NULL )  //输入元素有效
 {  q=(LNode*)malloc(sizeof(LNode));
    q->data=key;
    n=H(key);   //查HASH表得到指针数组的地址
    q->next=T[n];   //原来的头指针后移
    T[n]=q;         //新元素插入头部
 } //while
    return OK;
```

前页算法用的是尾插入方式，其中for循环和冲突处理方法非常出色。但是链表的建立过程中，逆向插入本来就相对简单，在该程序中对结点顺序又没有特殊要求，所以，头插入更加合适，无论是代码的书写，还是算法时间和空间效率分析上，都具有明显优越性。

3、再哈希法（双哈希函数法）

$H_i=RH_i(key)$　　$i=1, 2, \ldots, k$

RH_i均是不同的哈希函数，当产生冲突时就计算另一个哈希函数，直到冲突不再发生。

优点：不易产生聚集；

缺点：增加了计算时间。

4. 建立一个公共溢出区

思路：除设立哈希基本表外，另设立一个溢出向量表。

所有关键字和基本表中关键字为同义词的记录，不管它们由哈希函数得到的地址是什么，一旦发生冲突，都填入溢出表。

四、哈希表的查找及分析

明确：散列函数没有“万能”通式（杂凑法），要根据元素集合的特性而分别构造。

算法：见教材P259

讨论：哈希查找的速度是否为真正的O（1）？

不是。由于冲突的产生，使得哈希表的查找过程仍然要进行比较，仍然要以平均查找长度ASL来衡量。

一般地，ASL依赖于哈希表的装填因子α,它标志着哈希表的装满程度。

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}}$$

$0 \le \alpha \le 1$

α越大，表中记录数越多，说明表装得越满，发生冲突的可能性就越大，查找时比较次数就越多。

讨论：

1） 散列存储的查找效率到底是多少?

答：ASL与装填因子α有关！既不是严格的O(1)，也不是O(n)

$$ASL \approx 1 + \frac{\alpha}{2} \qquad (拉链法)$$

$$ASL \approx \frac{1}{2}(1 + \frac{1}{1-\alpha}) \qquad (线性探测法)$$

$$ASL \approx -\frac{1}{\alpha}\ln(1-\alpha) \qquad (随机探测法)$$

应尽量选择一个合适的α，以降低ASL的长度

2） “冲突”是不是特别讨厌?

答：不一定！正因为有冲突，使得文件加密后无法破译！（单向散列函数不可逆，常用于数字签名和间接加密）。

利用了哈希表性质：源文件稍稍改动，会导致哈希表变动很大。

典型应用：md5散列算法

md5散列算法——信息-摘要算法（1991年）

message-digest algorithm）——用于加/解密和数字签名

md5的典型应用是对一段信息（message）产生一个128位的信息摘要（message-digest），以防止被篡改。

例： md5用于网络（如BBS）登录时的身份认证

在BBS服务器上，用户的密码都是以md5（或其它类似的算法）经加密后存储在文件系统中。

当用户登录的时候，系统把用户输入的密码计算成md5值，然后再去和预存在服务器中的md5值进行比较，进而确定输入的密码是否正确。

优点：系统在不知用户Password明码的情况下就可以确定其登录的合法性。这不但可以避免用户的密码被具有系统管理员权限的用户知道，而且还在一定程度上增加了密码被破解的难度。

科普小知识：Hash函数与数字签名（数字手印）

HASH函数，又称杂凑函数，是在信息安全领域有广泛和重要应用的密码算法，它有一种类似于指纹的应用。

在网络安全协议中，杂凑函数用来处理电子签名，将冗长的签名文件压缩为一段独特的数字信息，像指纹鉴别身份一样保证原来数字签名文件的合法性和安全性。例如SHA-1和MD5都是目前最常用的杂凑函数。经过这些算法的处理，原始信息即使只更动一个字母，对应的压缩信息也会变为截然不同的“指纹”，这就保证了经过处理信息的唯一性。为电子商务等提供了数字认证的可能性。

安全的杂凑函数在设计时必须满足两个要求：

其一是寻找两个输入得到相同的输出值在计算上是不可行的（即抗碰撞）；

其二是找一个输入，能得到给定的输出在计算上是不可行的，即不可从结果推导出它的初始状态。

现在使用的重要计算机安全协议，如SSL，PGP都用杂凑函数来进行签名，一旦找到两个文件可以产生相同的压缩值，就可以伪造签名，给网络安全领域带来巨大隐患。

MD5就是这样一个在国内外有着广泛的应用的杂凑函数算法，它曾一度被认为是非常安全的。

然而，王小云教授04年发现，可以很快的找到MD5的“碰撞”，就是两个文件可以产生相同的“指纹”。这意味着，当你在网络上使用电子签名签署一份合同后，还可能找到另外一份具有相同签名但内容迥异的合同，这样两份合同的真伪性便无从辨别。05年破解SHA-1算法。

王小云教授的研究成果证实了利用MD5算法的碰撞可以严重威胁信息系统安全，这一发现使目前电子签名的法律效力和技术体系受到挑战。

王小云在密码学中做了开创性贡献，她的创新性密码分析方法揭示了被广泛使用的密码哈希函数的弱点，促进了新一代密码哈希函数标准。

设计了我国哈希函数标准SM3 ，在金融、交通、国家电网等重要经济领域广泛使用, 并于2018年10月正式成为ISO/IEC国际标准

本章小结

重点：**各种查找法的基本思想和算法实现**

主要算法：

静态查找法——顺序查找、折半查找、分块查找

动态查找法——二叉排序树查找

散列表查找——解决冲突：开地址和链地址法

难点：**二叉排序树的删除算法**

下周三交第9章查找的作业　**9.3　9.31　9.33**